DSA -- Advanced Algorithms (Graph + DP + Greedy)

[Target] Target: Explain Dijkstra, DP, Greedy algorithms in 40s. Know NP-Hard vs NP-Complete cold.
Read time: 25 minutes
INTERVIEW CRITICAL: Dijkstra WORKS on directed graphs (corrected from mock feedback)

MOCK FEEDBACK CORRECTION -- READ FIRST

Wrong answer given in mock: "Dijkstra does not work on directed graphs"
[OK] Correct answer: Dijkstra works on both directed AND undirected graphs.
The only restriction is: no negative edge weights.

1. DYNAMIC PROGRAMMING
Definition: Optimization technique that breaks problems into simpler sub-problems, stores results (memoization) to avoid redundant calculations.
Two key properties:
- Overlapping sub-problems: same sub-problems solved repeatedly
- Optimal substructure: optimal solution contains optimal solutions to sub-problems
DP Applications (ebook list):
- Multistage Graph
- Floyd-Warshall
- Bellman-Ford
- 0/1 Knapsack
- Optimal Binary Tree
- Reliability Design
- LCS (Longest Common Subsequence)
- Matrix Chain Multiplication

2. HUFFMAN CODING
What it is: Greedy algorithm for lossless data compression using variable-length encoding.
Key properties:
- Frequent characters shorter codes
- Rare characters longer codes
- Prefix rule: no code is a prefix of another code (avoids ambiguity during decoding)
- Uses variable-length encoding
Algorithm Steps:
1. Create leaf node for each character (with frequency)
2. Arrange all nodes in ascending order of frequency
3. Take two nodes with lowest frequency create new internal node (sum of both)
4. Repeat steps 2-3 until single tree formed
5. Assign codes: traverse tree -- Left = 0, Right = 1
Time Complexity:
- ExtractMin() called 2(n-1) times
- Each call = O(log n) via min-heap
- Total: O(n log n)
Worked Example from ebook:
Text: ABBCDBCCDAABBEEEBEAB

Char
Frequency
Code

A
4
01

B
7
11

C
3
101

D

3
101

E
4
00

Huffman Tree:

40s Script:

"Huffman coding is a greedy algorithm for lossless data compression. It assigns variable-length codes based on character frequency -- frequent characters get shorter codes. We build a binary tree bottom-up by repeatedly combining the two nodes with lowest frequencies. The prefix rule ensures no code is a prefix of another, making it uniquely decodable. Time complexity is O(n log n)."

3. KNAPSACK PROBLEM
Type 1 -- Fractional Knapsack (Greedy)

Items are divisible -- can take fractions
Strategy: sort by value/weight ratio (descending), take greedily
Always gives optimal solution
Time complexity: O(n log n) if already sorted, else O(n log n) for sort

Type 2 -- 0/1 Knapsack (Dynamic Programming)

Items are indivisible -- take whole or nothing
Greedy does NOT work here -- DP required
Formula: V[i, w] = max{V[i-1, w], V[i-1, w-wi] + Pi}
Time complexity: O(n W)

Solving 0/1 Knapsack:
1. Draw table T with (n+1) rows and (W+1) columns
2. Fill row 0 and column 0 with zeros
3. Fill row-wise from top to bottom, left to right
4. Last box = maximum possible value
5. Backtrack to find which items were selected
Worked Example (ebook):
- Capacity W=8, n=4 items
- Profits P = {1, 2, 5, 6}, Weights w = {2, 3, 4, 5}
Constraint: sum xi-wi <= m | Objective: max sum xi-Pi

4. MULTISTAGE GRAPH
A directed weighted graph where vertices are divided into stages. Find minimum cost path from source (S) to destination (D).

Formula: Fun(Si, Vj) = min{Fun(Si+1, K) + C(Vj, K)}
Time Complexity: O(V + E)

5. SHORTEST PATH ALGORITHMS
A. DIJKSTRA'S ALGORITHM INTERVIEW CRITICAL
Definition: Single-source shortest path for graphs with non-negative edge weights. Greedy approach.

Property
Value

Works on
Both directed AND undirected graphs [OK]

Negative weights
Does NOT work

Time complexity
O(E log V) with min-heap

Space complexity
O(V)

Application
Google Maps, routing protocols

Algorithm Steps:
- Step 1: Define two sets -- visited (shortest path found) and unvisited (remaining)
- Step 2: Initialize: d[source] = 0, d[all others] = , [all] = NIL
- Step 3: Extract vertex u with minimum d[u]. Relax all outgoing edges.
- Edge relaxation: if d[u] + w(u,v) < d[v], then update d[v] = d[u] + w(u,v)
40s Script:

"Dijkstra finds single-source shortest paths in weighted graphs with non-negative edge weights. It's greedy -- maintains a set of visited vertices and greedily picks the unvisited vertex with minimum current distance. For each extracted vertex, it relaxes all outgoing edges. Works on both directed and undirected graphs. Fails with negative weights because the greedy assumption breaks -- a later path through a processed vertex could be shorter."

B. BELLMAN-FORD ALGORITHM
Key difference from Dijkstra: Handles negative edge weights and detects negative cycles.
Algorithm:
- Relax ALL edges (n-1) times (n = number of vertices)
- If after (n-1) iterations, still can relax negative cycle detected
Edge relaxation:
if d[u] + c(u,v) < d[v]:
    d[v] = d[u] + c(u,v)

Property
Dijkstra
Bellman-Ford

Negative weights

[OK]

Negative cycles

[OK] Detects

Time complexity
O(E log V)
O(E V) = O(n)

Worked Example (ebook):
Vertices: A-F, Edges with negative weights
- A-0, B-1, C-3, D-5, E-0, F-3

C. FLOYD-WARSHALL ALGORITHM
Purpose: All-pairs shortest path (not just single source)
Strategy: Dynamic Programming -- try every vertex as intermediate
Formula:
A^k[i, j] = min{ A^(k-1)[i, j], A^(k-1)[i, k] + A^(k-1)[k, j] }
Time Complexity: O(n)

Space Complexity: O(n)

6. GREEDY ALGORITHMS
Makes locally optimal choice at each step hoping to achieve global optimum.
When greedy works: MST, Huffman, Dijkstra, Fractional Knapsack, Optimal Merge Pattern
When greedy fails: 0/1 Knapsack, TSP, most optimization problems
KRUSKAL'S ALGORITHM (MST)
Minimum Spanning Tree of a weighted, connected, undirected graph.
Steps:
1. Draw all vertices
2. Sort all edges by weight (ascending)
3. Add edges one by one -- skip if adding creates a cycle
4. Stop when n-1 edges added
Time Complexity: O(E log V) or O(E log E)
Example (ebook): Graph with vertices A,B,C,D,E
- Add edges in order: 1, 2, 4, 5, 6 (skip if cycle)
- Weight of MST = 1+2+4+5+6 = 18

7. MATRIX CHAIN MULTIPLICATION
Problem: Given n matrices, find the optimal order to multiply them to minimize total scalar multiplications.
Formula:
M[i, j] = min{ M[i, k] + M[k+1, j] + P(i-1) P(k) P(j) }
for all i <= k < j
Time Complexity: O(n)

8. DIVIDE AND CONQUER
Break problem into sub-parts, solve recursively, combine results.
Applications:
- Binary Search
- Merge Sort
- Quick Sort
- Strassen's Matrix Multiplication
- Karatsuba Algorithm

9. TRAVELING SALESMAN PROBLEM (TSP)

Visit every city exactly once and return to start
Minimum cost Hamiltonian cycle
NP-Complete problem
Solved by DP: g(i, S) = min{C(ik) + g(k, S-{k})}

10. HAMILTONIAN CYCLE

Path that visits every vertex exactly once and returns to start
Max nodes = (n-1)!/2
Complexity: T(n) = O(n)

Rules:
- Every vertex i and (i+1) must be adjacent
- First and last vertex must be adjacent
- Vertex i must not repeat in first (i-1) positions

11. BACKTRACKING
Brute force approach -- explore all possible solutions, prune dead ends.
Examples:
- N-Queens: place N queens on N N board, no two attack each other
  - Time: O(N!), Space: O(N)
- Graph Coloring: color vertices so no adjacent vertices share same color
  - Chromatic number = minimum colors needed
  - Nodes = C^(n+1)
- Hamiltonian Cycle: visit each vertex exactly once

12. BRANCH AND BOUND
Algorithmic technique for hard optimization problems.

Guarantees optimal solution will be found
Models solution space as a tree
Intelligently eliminates branches that cannot improve current best
Does NOT guarantee polynomial time

Job Sequencing with Deadline:
- Maximize profit by scheduling jobs before their deadlines
- U = sum Pi (all penalties) | C = sum Pi (running cost)

13. NP-HARD AND NP-COMPLETE

Term	Definition
P	Solvable in polynomial time
NP	Solution verifiable in polynomial time
NP-Hard	At least as hard as hardest NP problem
NP-Complete	Both NP and NP-Hard

Polynomial time examples:
- Linear search O(n)
- Binary search O(log n)
- Insertion sort O(n)
- Merge sort O(n log n)
- Matrix multiplication O(n)

Exponential time / NP examples:
- 0/1 Knapsack: 2
- Traveling Salesman: 2
- Sum of subsets: 2
- Graph coloring: 2
- Hamiltonian cycle: 2

Key rule:
- If NP-Hard or NP-Complete is solved in polynomial time NP = P
- Currently: NP != P (believed, not proven)

Lower Bound Theory:
L(n) >= C * g(n), for n > n
g(n) is the lower bound of algorithm A.

Quick Cheat Sheet

Huffman	greedy, lossless compress, O(n log n), prefix rule
Knapsack	Fractional=greedy(value/weight ratio), 0/1=DP O(nW)
Multistage	DP, min cost path through stages, O(V+E)
Dijkstra	single source, non-negative weights, O(E log V), directed+undirected [OK]
Bellman-Ford	handles negative weights, detects negative cycles, O(VE)
Floyd-Warshall	ALL pairs shortest path, DP, O(n)
Kruskal's	MST, sort edges, skip cycles, O(E log E)
Matrix Chain	DP, optimal multiply order, O(n)
Divide&Conquer	BinarySearch, MergeSort, QuickSort, Strassen
TSP	NP-Complete, visit all cities once, DP: O(n 2)
Backtracking	N-Queens, Graph Coloring, Hamiltonian
Branch&Bound	optimization, prune solution tree, job scheduling
NP-Complete	TSP, Hamiltonian, Graph Coloring, Knapsack, Sum-of-subsets

Follow-Up Questions
Q: Why doesn't Dijkstra work with negative weights?

The greedy assumption breaks -- once a vertex is "settled" (minimum distance found), we assume it's final. With negative weights, a future path through an already-settled vertex could be shorter. Bellman-Ford handles this by re-relaxing all edges (n-1) times.

Q: Difference between Kruskal's and Prim's algorithm for MST?

Kruskal's sorts all edges globally and adds the cheapest safe edge (edge-based). Prim's grows the MST from a starting vertex by always adding the minimum weight edge connected to current tree (vertex-based). Both give optimal MST.

Q: When to use DP vs Greedy?

Greedy: when local optimal choices lead to global optimum (Huffman, Dijkstra, Fractional Knapsack). DP: when sub-problems overlap and greedy fails (0/1 Knapsack, Floyd-Warshall, Matrix Chain). If greedy gives wrong answer on a small example, switch to DP.

Q: What is the difference between NP-Hard and NP-Complete?

NP-Complete = NP + NP-Hard (it is both verifiable in polynomial time AND as hard as any NP problem). NP-Hard problems are at least as hard as NP-Complete but may not even be in NP (their solutions may not be verifiable in polynomial time -- example: Halting Problem).

HIGH PRIORITY TOPICS
1. DYNAMIC PROGRAMMING
Definition:
- Optimization technique that breaks complex problems into simpler sub-problems
- Stores results of sub-problems to avoid redundant calculations (memoization)
- Bottom-up approach: solve smaller problems first, build to larger ones
Key Properties:
1. Overlapping sub-problems: same sub-problems solved multiple times
2. Optimal substructure: optimal solution contains optimal solutions to sub-problems
Types:
- Multistage Graph: weighted directed graph with stages, minimum cost path
- Floyd-Warshall: all-pairs shortest path algorithm
- 0/1 Knapsack: include/exclude items to maximize value within weight constraint
- Optimal Binary Tree: minimize search cost for frequency-based searches
Time Complexity: Generally O(n) to O(n)

2. HUFFMAN CODING
Definition:
- Lossless data compression algorithm using variable-length encoding
- Assigns shorter codes to frequent characters, longer codes to rare characters
Algorithm Steps:
1. Build frequency table for all characters
2. Create leaf nodes, arrange in ascending order of frequency
3. Make new internal node with two lowest frequency nodes as children
4. Repeat until single tree formed
Properties:
- Prefix property: no code is prefix of another code
- Optimal: guarantees minimum average code length
- Time Complexity: O(n log n) using min-heap
40s Script:
"Huffman coding is a greedy algorithm for lossless compression. It assigns variable-length codes based on character frequency - frequent characters get shorter codes. We build a binary tree bottom-up by repeatedly combining the two nodes with lowest frequencies. The path from root to leaf gives each character's code. This ensures no code is a prefix of another, making it uniquely decodable."

3. KNAPSACK PROBLEM
Types:
1. Fractional Knapsack: can take fractions of items (greedy works)
2. 0/1 Knapsack: can only take whole items (DP required)
0/1 Knapsack Solution:
```
If items are already arranged by value/weight ratio:
V[i,w] = max{V[i-1,w], V[i-1,w-wi] + vi}
Where: V[i,w] = maximum value with first i items and weight limit w
```
Formula:
- Constraint: sum wi <= W (total weight <= capacity)
- Objective: max sum vi (maximize total value)
Time Complexity: O(nW) where n = items, W = capacity

4. SHORTEST PATH ALGORITHMS
A. DIJKSTRA'S ALGORITHM INTERVIEW CRITICAL
Definition:
- Single-source shortest path for non-negative weighted graphs
- Uses greedy approach with priority queue (min-heap)
Key Properties:
- [OK] Works on directed AND undirected graphs
- Does NOT work with negative weights
- Time Complexity: O((V+E) log V) with heap
Algorithm Steps:
1. Initialize distance array: d[source] = 0, others =
2. Add all vertices to min-priority queue
3. Extract vertex u with minimum distance
4. For each neighbor v of u: if d[u] + weight(u,v) < d[v], update d[v]

5. Repeat until queue empty
40s Script:
"Dijkstra finds shortest paths from a source to all vertices in weighted graphs with non-negative edges. It's a greedy algorithm using a priority queue. We start with source distance 0, others infinity. Repeatedly extract the minimum distance vertex and relax its neighbors. Works on both directed and undirected graphs but fails with negative weights."
B. BELLMAN-FORD ALGORITHM
Key Difference from Dijkstra:
- [OK] Handles negative weights
- [OK] Detects negative cycles
- Slower: O(VE) time complexity
Algorithm:
- Relax all edges (V-1) times
- Check for negative cycles in Vth iteration
C. FLOYD-WARSHALL ALGORITHM
Purpose: All-pairs shortest path
Time Complexity: O(V)
Formula: A^k[i,j] = min{A^(k-1)[i,j], A^(k-1)[i,k] + A^(k-1)[k,j]}

5. GREEDY ALGORITHMS
Definition:
- Makes locally optimal choice at each step
- Does not guarantee global optimum for all problems
- Works for problems with greedy choice property
Classic Examples:
1. Minimum Spanning Tree: Kruskal's, Prim's
2. Single Source Shortest Path: Dijkstra's
3. Huffman Coding
4. Fractional Knapsack
5. Optimal Merge Pattern
KRUSKAL'S ALGORITHM (MST)
Steps:
1. Sort all edges by weight (ascending)
2. Add edges one by one if they don't create cycle
3. Use Union-Find to detect cycles
Time Complexity: O(E log E)

6. COMPLEXITY CLASSES
NP-HARD and NP-COMPLETE
P: Problems solvable in polynomial time
NP: Non-deterministic polynomial time (can verify solution in polynomial time)
NP-Hard: At least as hard as any problem in NP
NP-Complete: Both NP and NP-Hard
Examples:
- NP-Complete: Traveling Salesman, Hamiltonian Cycle, Graph Coloring
- NP-Hard: Halting Problem
40s Script:
"P problems can be solved in polynomial time. NP problems can have their solutions verified in polynomial time. NP-Complete problems are both in NP and as hard as any NP problem - if we solve one in polynomial time, we solve all. NP-Hard problems are at least as difficult as NP-Complete but might not be in NP themselves."

7. BACKTRACKING
Definition:
- Brute force approach that systematically explores solution space
- Abandons partial solutions that cannot lead to complete solution
Examples:
- N-Queens Problem: Place N queens on NxN chessboard so none attack
- Graph Coloring: Color graph vertices so no adjacent vertices same color
- Hamiltonian Cycle: Visit each vertex exactly once and return to start
Template:
```
if (solution found):
    return solution
if (no valid extension):
    return failure
for each extension:
    try extension
    if (backtrack succeeds):
        return solution
    undo extension
```

Quick Cheat Sheet
```
Dynamic Programming:
  0/1 Knapsack      V[i,w] = max{V[i-1,w], V[i-1,w-wi] + vi}
  Floyd-Warshall    A^k[i,j] = min{A^(k-1)[i,j], A^(k-1)[i,k] + A^(k-1)[k,j]}
Shortest Path:
```

```
  Dijkstra          Non-negative weights, O((V+E) log V), works on directed/undirected
  Bellman-Ford      Negative weights OK, O(VE), detects negative cycles
  Floyd-Warshall    All pairs, O(V )
Greedy Works For:
  MST (Kruskal, Prim)   Always optimal
  Huffman Coding        Always optimal
  Fractional Knapsack   Always optimal
  Dijkstra              Always optimal (non-negative weights)
NP Complexity:
  P          Polynomial solvable
  NP         Polynomial verifiable
  NP-Hard    As hard as any NP problem
  NP-Complete   NP + NP-Hard
```

Follow-up Questions
Q: Why doesn't Dijkstra work with negative weights?
A: Because the greedy choice (always picking minimum distance vertex) fails. Once a vertex is processed, we assume its shortest path is found. With negative weights, a later path might give a shorter route through an already-processed vertex.
Q: Difference between Kruskal's and Prim's?
A: Both find MST. Kruskal sorts all edges globally and adds safe edges (edge-based). Prim grows tree from a starting vertex by adding minimum weight edge connected to current tree (vertex-based).
Q: When to use DP vs Greedy?
A: Use DP when problem has overlapping sub-problems and optimal substructure, but greedy choice doesn't work globally. Use Greedy when local optimal choices lead to global optimum.